DSC 102: Systems for Scalable Analytics

PA2: Task Parallelism on Multiple Cores

Presenter: Qiyu Li

Slide Contributor: Arun Kumar, Vraj Shah

Goals

Get comfortable with datasets that are too big to fit in a single-node machine

Use Dask library to explore disk-aware data access and task parallelism



Perform data explorations to build intuitions for feature engineering for PA2

Dask: Scheduling

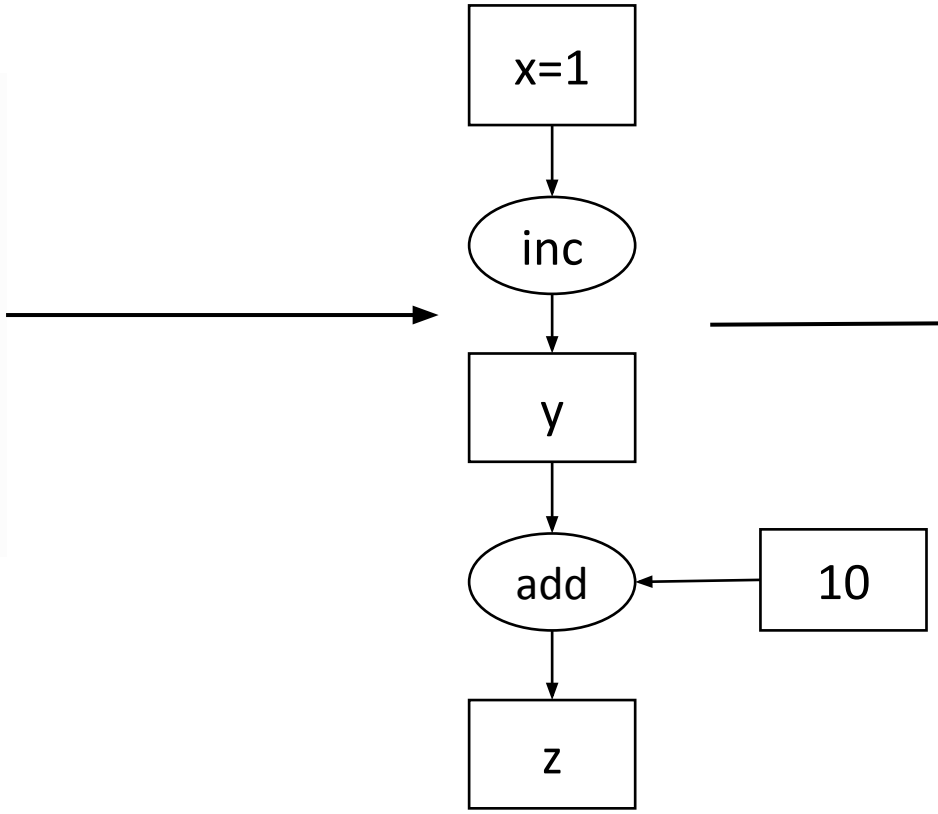
Dask APIs create task graphs and Dask schedulers execute them

```
def inc(i):
    return i + 1

def add(a, b):
    return a + b

x = 1
y = inc(x)
z = add(y, 10)
```

Collections



Task Graph

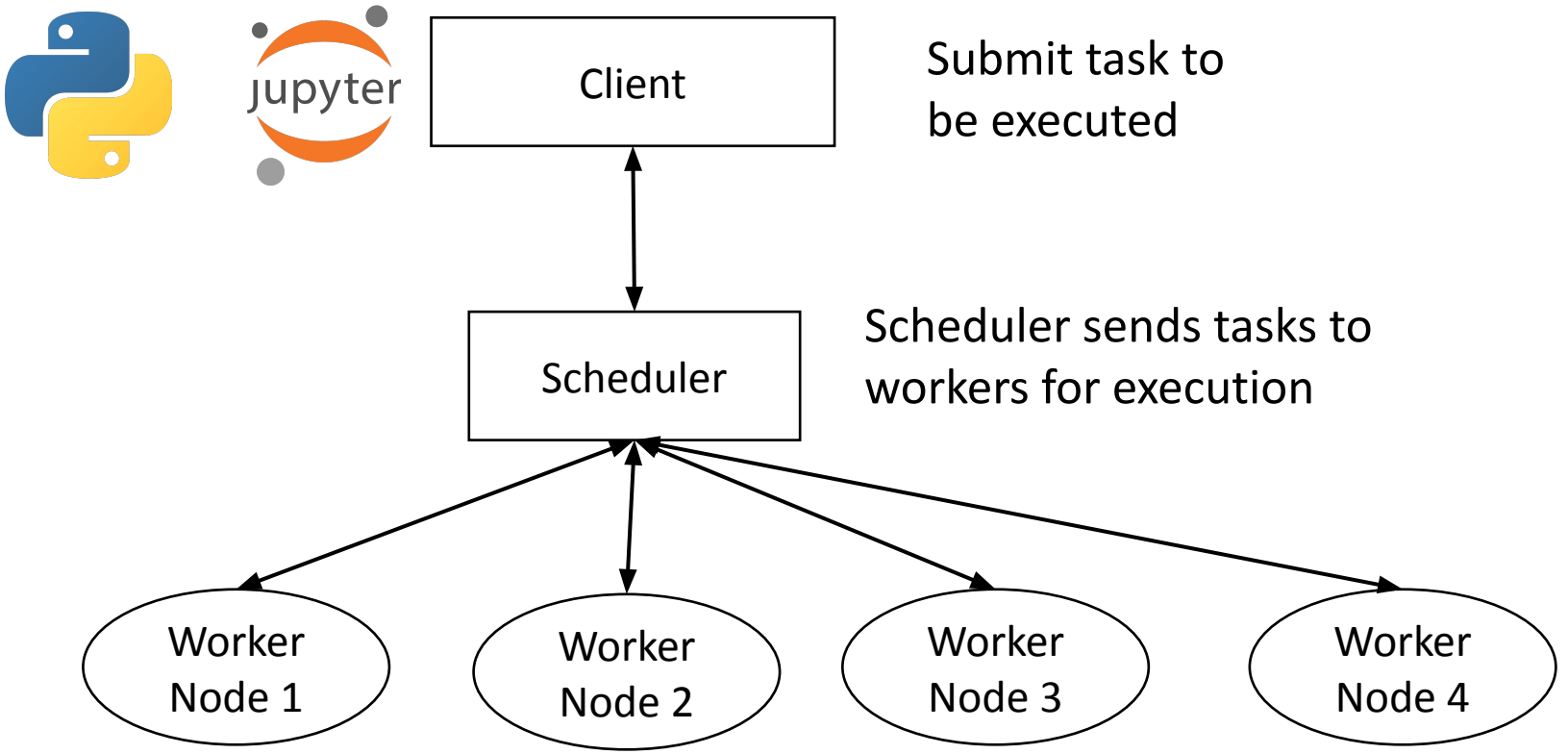
Single-machine scheduler

Distributed scheduler

We will use this in our assignment!

Schedulers

Dask: Distributed Execution



Assignment: Dataset Description

Column name	Example
reviewerID	A32DT10X9WS4D0
asin	B003VX9DJM
reviewerName	Slade
helpful	[4, 5]
reviewText	this was a gift for my friend who loves touch lamps.
overall	1
summary	broken piece
unixReviewTime	1397174400
reviewTime	04 11, 2014

Reviews table

Column name	Example
asin	143561
title	Everyday Italian (with Giada de Laurentiis)
description	3Pack DVD set - Italian Classics
price	12.99
imUrl	http://g-ecx.images-amazon.com /31mC.jpg
brand	1
categories	[['Movies & TV', 'Movies']]
related	{'also_viewed': ['B0036FO6SI', '000014357X'],'buy_after_viewing': ['B0036FO6SI', 'B000KL8ODE']}
salesRank	{'Movies & TV': 376041}

Products table

Assignment Tasks: PA 2

Compute descriptive stats and perform data consistency checks using both *reviews* and *products* table

Task No.	**Task Description**
1	% Missing values for all columns in the reviews table
2	% Missing values for all columns in the product table
3	Pearson correlation between rating and price
4	Descriptive stats for price
5	Number of products for each super-category
6	Check for dangling reference of the product ids in the reviews table
7	Check for dangling reference of the product ids in the related column of the products table

Reminder: PA 2

If there is a value V in column X of table A, but this value is missing in column X of table B, we say that V is a dangling reference of X from A to B.

Task No.	Task Description
6	Check for dangling reference of the product ids in the reviews table
7	Check for dangling reference of the product ids in the related column of the products table

Example:

If there is a value V in column X of table A, but this value is missing in column X of table B, we say that V is a dangling reference of X from A to B.

Which of the below is dangling?

- **Products_Table** | product_id | product_name | | :--- | :--- | | P101 | "Laptop" | | P102 | "Mouse" | | P103 | "Keyboard" |

  Reviews_Table | review_id | product_id | | :--- | :--- | | R1 | P101 | | R2 | P102 | | R3 | P101 |

- **Products_Table** | product_id | product_name | | :--- | :--- | | P201 | "Coffee" | | P202 | "Tea" | | P203 | "Sugar" |

  Reviews_Table | review_id | product_id | | :--- | :--- | | R4 | P201 | | R5 | P201 |

- **Products_Table** | product_id | product_name | | :--- | :--- | | P301 | "Book" | | P302 | "Pen" |

  Reviews_Table | review_id | product_id | | :--- | :--- | | R6 | P301 | | R7 | P303 |

Grading Scheme

Accuracy (80%)

Task No.	Task Description	Points
1	% Missing values for all columns in *reviews* table	6
2	% Missing values for all columns in *products* table	6
3	Pearson correlation between rating and price	14
4	Descriptive stats for price	12
5	Number of products for each super-category	14
6	Check for dangling reference of product ids from *reviews* table to *products* table	14
7	Check for dangling reference of product ids in the *related* column to *asin* column of *products* table	14

We tolerate 1% error margin, but solutions should match exactly on element-wise basis to ours

No partial markings!

Grading Scheme

Runtime (20%)

Absolute 4 node runtimes (5)	**Points**
Under 5 mins	5
Between 5 mins to 10 mins	3
Between 10 mins to 30 mins	2
Anything above 30 mins	0

Runtime speedup with 4 nodes (5)	**Points**
Greater than 3.5x	5
Between 2.5x to 3.5x	3
Between 1x to 2.5x	2
Under 1x	0

Extra Credit (10) : Part A full accuracy points and runtime <= our solution's runtime

PA System Setup notes

You will run your code on two different cluster sizes (2 instances and 5 instances) to measure and evaluate speedup.

Key Components:



- Dask Scheduler: The "coordinator" process that manages tasks.
- Dask Workers: The "executor" processes that execute tasks.

Differences from PA1

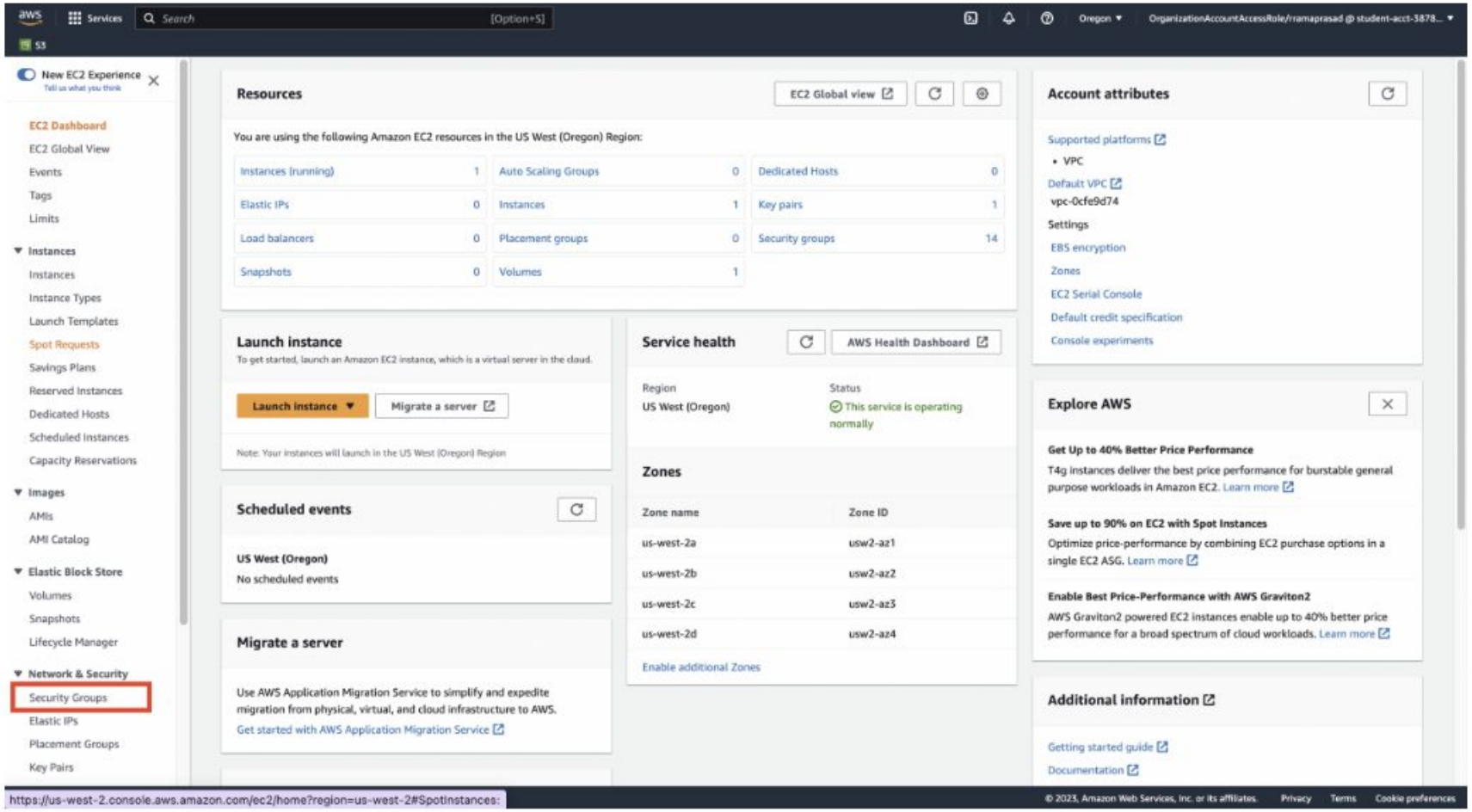
Instances: You will launch **5 instances at once**, not 1.

- Instance 1: Will run the Dask Scheduler & Jupyter Notebook.
- Instances 2-5: Will run the Dask Workers.

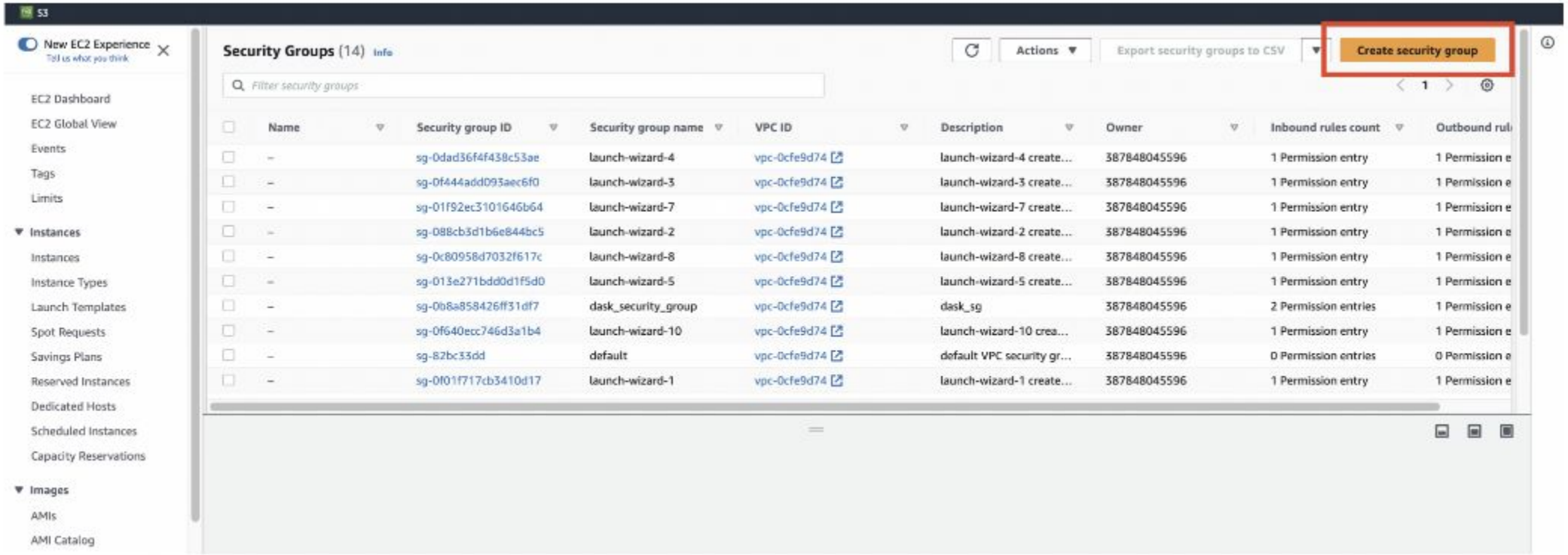
Each of the 5 instances needs **100GB of SSD storage** (up from 40GB).

You must create a **new Security Group** that allows all 5 instances to freely communicate with each other.

Security group



Security group



Launch Instances

A. Give any name for your instance.

b. Number of instances to launch is 5.

c. The instance type is ”t2.xlarge”.

d. Create a new key pair for SSH’ing to your instance later. The private key will be downloaded to your local machine, similarly to PA0

Launch Instances

- Under ”Network Settings”, click ”**Select Existing Security Group**” and choose the name of the security group you just created, e.g., ”dask security group”.
- Choose **100GB SSD** gp2 storage.
- Open advanced details. Select ”Request Spot Instances”.
- Lastly, click ”Launch Instance"

Key Info for PA2

- You will be expected to use several pieces of dask functionality that you didn't in PA1.

- **dask.compute()** for **Parallel Graph Execution**
  - In PA1, you did this once at the end (hopefully)

- In PA2, you have 5+ independent questions (Q1, Q2, Q3, Q4, Q5, etc.).
  - Instead of running ans1.compute(), then ans2.compute() sequentially, dask.compute() runs them all in parallel, which is much more efficient.

Key Info for PA2

- Broadcasting a set for Efficient Lookups
- compute a small, "global" piece of data (like a set of all valid IDs) and then Dask efficiently sends this data to all worker nodes.
- For Q6 and Q7 (dangling reference checks), you need to check if millions of product IDs exist in your products table
  - The fastest way to do this is to compute the set of all valid product asins once and use it as a lookup table.

Key Info for PA2

- .apply() with ast.literal_eval
- Apply/Map are faster than row-wise (can be done in parallel)
- categories: [['Movies & TV', 'Movies']]
- related :{'also_viewed': ['B0036FO6SI', '000014357X’],'buy_after_viewing': ['B0036FO6SI', 'B000KL8ODE']}
- Ast will let you parse these as strings and convert to dicts

Key Info for PA2

- Use .map_partitions(func) for parallelism
  - This will run a function *in parallel* on your code
  - Remember, you only need to find *one dangling reference* for Q6

Key Info for PA2

- Use functors where you can (lambdas in python)

  Loop:

```
numbers = [1, 2, 3]
doubled_list = []
for x in numbers:
    doubled_list.append(x * 2)
```

  Functor:

```
numbers = [1, 2, 3]
def double(x): return x * 2

doubled_list = list(map(double, numbers))
```

Tips for PA2

- Assign values to the questions q1, q2, q3 … etc.
- Look into using dask merge to make an overall dataframe for some calculations.
- Make sure to check your dashboard to see if you all your workers are active and being used.
- Terminate EC2 instances once you are done.
- Remember when you terminate an EC2 instance you lose all the data, therefore we suggest you use a private GitHub repo to routinely push your work (code, logs and other small files) and pull your repo whenever you create a new instance to resume your work.

Setup Demo